

Lab 2 - Week 4. Invariant and Relations

Aim

Apply the theory of Invariants and relations to practical problems using Haskell.

Prerequisites

- Read or reread Chapters 4 and 5 of *The Haskell Road*.
- Make a list of questions on specific points that cause difficulty in understanding.

Submission guidelines

Files

- Submit one Haskell file per exercise.
- Name files ExerciseX.hs for exercises and EulerX.hs for Euler problems.
- Each module must be named to match: module ExerciseX where.
- Follow naming conventions exactly — some exercises are auto-graded.
- Do not include personally identifiable information.
- Indicate additional dependencies in a comment at the top of the file.
- Record time spent:
 - Time Spent: X min (in minutes).
- Codegrade: create a new group with all teammates for each assignment.
- Include a txt file containing the report on design choices
- You are not allowed to use GenAI to immediately obtain a solution to the proposed exercises. Any other use of GenAI is subject to the UvA AI policy (<https://www.uva.nl/en/about-the-uva/policy-and->

[regulations/education/policy-framework-and-guidelines-on-genai-in-education.html](https://www.gov.uk/government/policy-framework-and-guidelines-on-genai-in-education.html)).

Imports

```
import Data.List
import System.Random
import Test.QuickCheck
import SetOrd
```

Exercise 1

Implement a random data generator for the datatype `Set Int`, where `Set` is as defined in `SetOrd.hs`. First do this from scratch, next give a version that uses *QuickCheck* to random test this datatype.

Deliverables: two random test generators, oral and written design report, test report, indication of time spent.

Exercise 2

Implement operations for set intersection, set union and set difference, for the datatype `Set` defined in `SetOrd.hs`. Next, use automated testing to check that your implementation is correct. First use your own generator, next use *QuickCheck*.

Use the following declarations:

```
setIntersection :: Ord a => Set a -> Set a -> Set a
setUnion       :: Ord a => Set a -> Set a -> Set a
setDifference  :: Ord a => Set a -> Set a -> Set a
```

Deliverables: implementations, test properties, oral and written design report, test report, indication of time spent.

Exercise 3

Suppose we implement binary relations as list of pairs, Haskell type `[(a,a)]`. Assume the following definition:

```
type Rel a = [(a,a)]
```

Use the following declaration:

```
symClos :: Ord a => Rel a -> Rel a
```

to define a function that gives the symmetric closure of a relation, where the relation is represented as an ordered list of pairs. E.g., `symClos [(1,2),(2,3),(3,4)]` should give `[(1,2),(2,1),(2,3),(3,2),(3,4),(4,3)]`.

Deliverables: Haskell program, brief discussion on the implementation (with definition of symmetric closure), oral and written design report, indication of time spent.

Exercise 4

A relation R is serial on a domain A if for any $x \in A$ there is an $y \in A$ such that xRy . Suppose relations are represented as lists of pairs:

```
type Rel a = [(a,a)]
```

1. Write a function for checking whether a relation is serial. This function accepts the domain as the first parameter and the `Rel` as the second parameter.

```
isSerial :: Eq a => [a] -> Rel a -> Bool
```

2. Test your implementation with two QuickCheck properties.
3. Consider the relation $R = \{(x, y) \mid x \equiv y \pmod{n}\}$, where $\pmod{n}$ is the modulo function in modular arithmetic and $n > 0$. Discuss whether (and when) R is serial. How can you test whether R is serial? How can you prove that R is serial?

Deliverables: Haskell program, QuickCheck properties, oral and written design report, test report (including the proof), indication of time spent.

Exercise 5

Use the datatype for relations from the previous exercise, plus

```
infixr 5 @@

(@@) :: Eq a => Rel a -> Rel a -> Rel a
r @@ s =
  nub [ (x,z) | (x,y) <- r, (w,z) <- s, y == w ]
```

to define a function:

```
trClos :: Ord a => Rel a -> Rel a
```

that gives the transitive closure of a relation, represented as an ordered list of pairs. E.g., `trClos [(1,2),(2,3),(3,4)]` should give `[(1,2),(1,3),(1,4),(2,3),(2,4),(3,4)]`.

Deliverables: Haskell program, brief discussion on the implementation (with definition of transitive closure), oral and written design report, indication of time spent.

Exercise 6

Test the functions `symClos` and `trClos` from the previous exercises. Devise your own test method for this. Try to use random test generation. Define reasonable properties to test. Can you use QuickCheck? How?

Deliverables: test code, oral and written design report, test report, indication of time spent.

Exercise 7

Is there a difference between the symmetric closure of the transitive closure of a relation R and the transitive closure of the symmetric closure of R ?

Hint: If your answer is that these are the same, you should give an argument, if you think these are different you should give an example that illustrates the difference.

Deliverables: Haskell file with the answer in comment form, indication of time spent

Exercise 8 (Bonus)

In the lecture notes, `Statement` is in class `Show`, but the `show` function for it is a bit clumsy. Write your own `show` function for imperative programs. Next, write a `read` function, and use `show` and `read` to state some abstract test properties for how these functions should behave. Next, use `QuickCheck` to test your implementations.

Deliverables: implementation, `QuickCheck` properties, oral and written design report, test report, indication of time spent.

Bonus 2

If this was all easy for you, you might wish to throw in a solution to a difficult problem (log in for the difficulty to be visible) from [Project Euler](#).
